PANDAS AND [GEOSPATIAL] DATA DISCOVERY

- DATA DATA EVERYWHERE.....IN WALKS PANDAS AND GEO-PANDAS
- PRACTICAL 7



David Topping: david.topping@manchester.ac.uk

'Vanilla' Python

Data types
Basic operations

Loops
Conditionals

Pandas and data discovery

Geospatial analysis

Statistical Analysis

Cluster Analysis

1 2 3 4 5 6 7 8 9

The Numpy module

Conway's Game Of Life

Differential Equations

Modules and Functions

Creating a notebook assessment
select topic from weeks 4 - 9

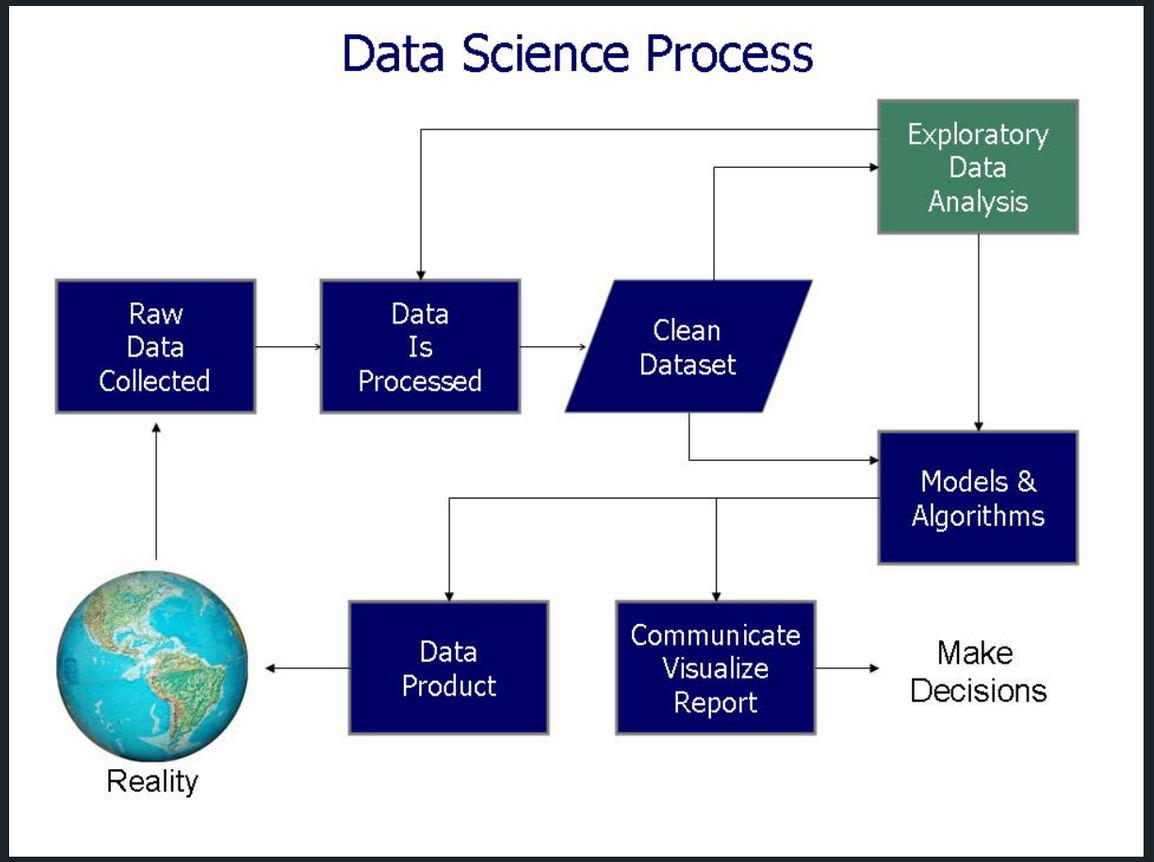
10 11 12

COURSE LAYOUT >>>>

LECTURE + PRACTICAL. SOLUTIONS ARE GIVEN

RECAP

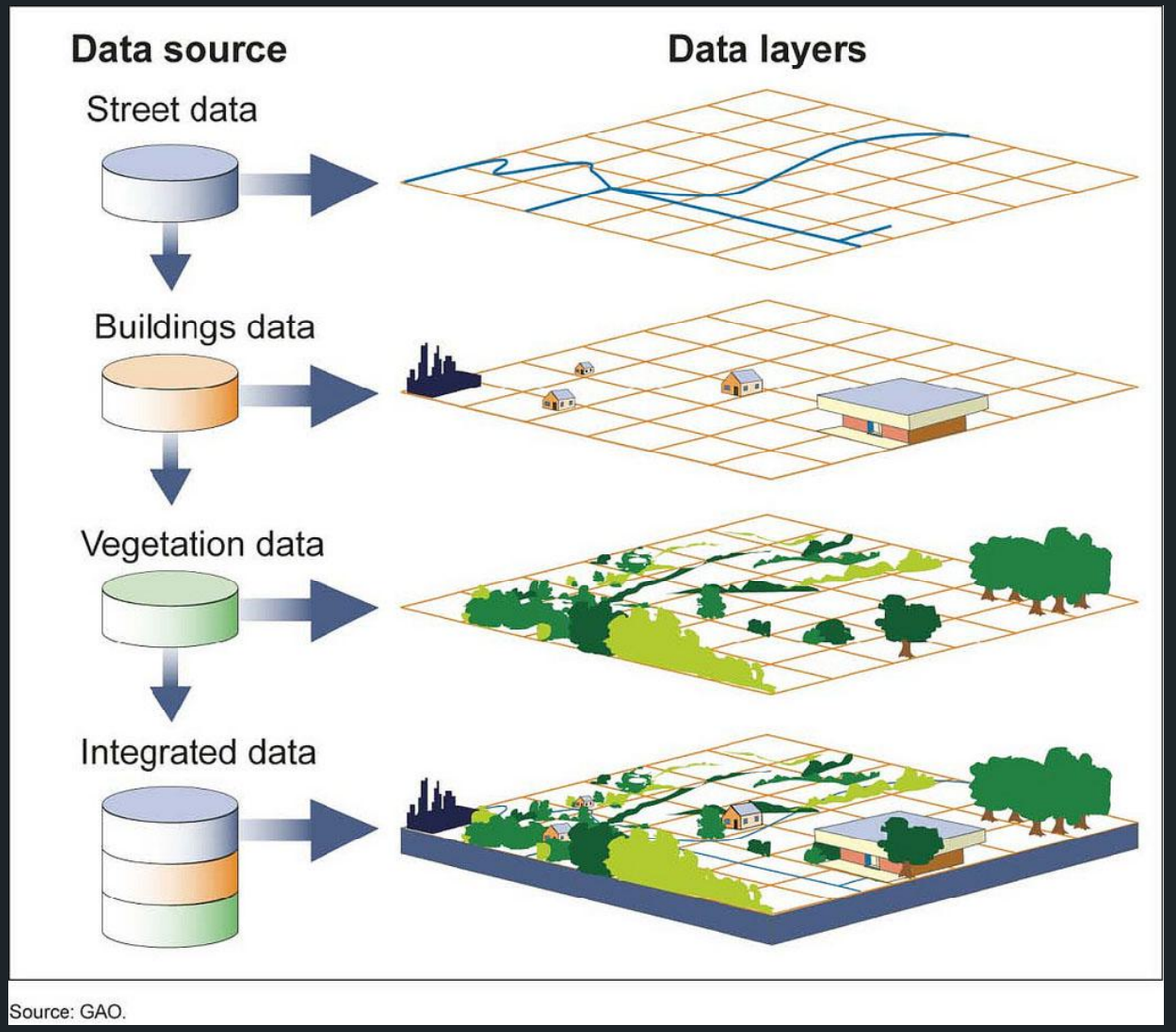
- Pandas allows us to store information in a dataframe
- Pandas can deal with various data types
- We can access entries in that dataframe according to certain criteria
- Dataframes have internal functions we can use to our advantage



What if we have data we want to perform statistical analysis on the data?

GEOGRAPHIC INFORMATION SYSTEMS (GIS)

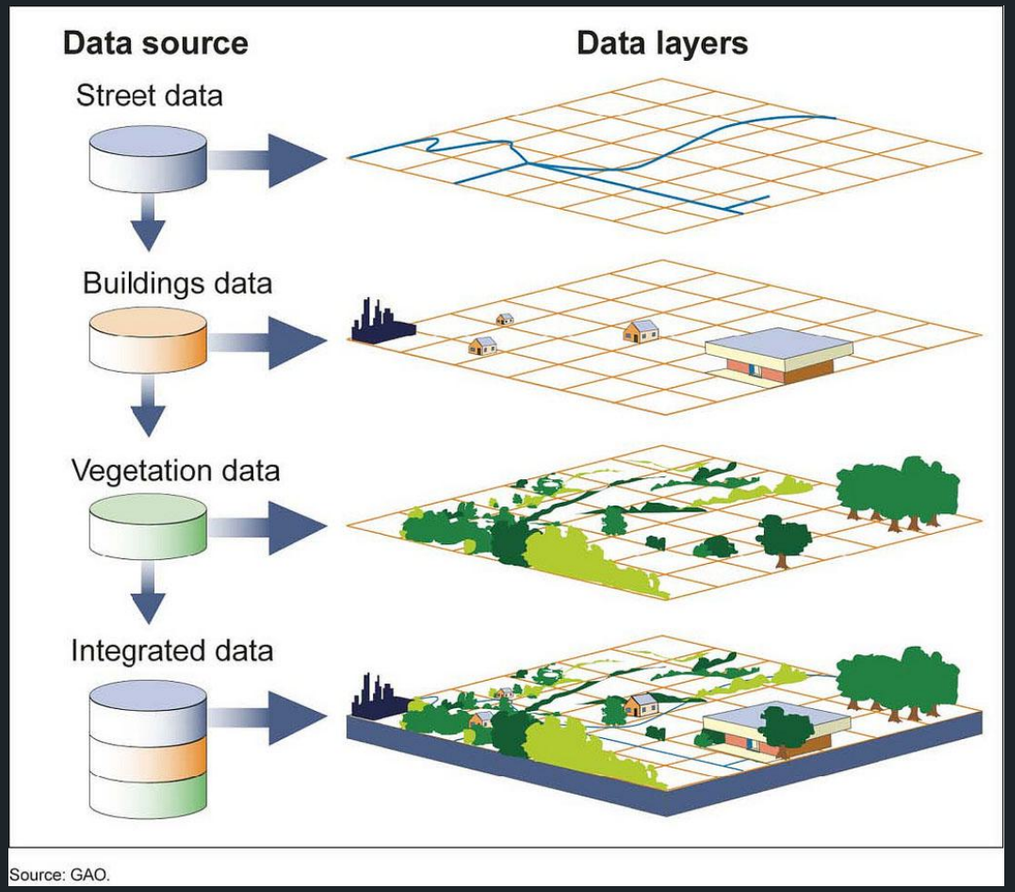
- **Geographic** refers to the physical location or position on the Earth's surface.
- **Information** refers to the data and details that can be used to represent and provide a better understanding of a specific location.
- **Systems** refer to a tool that makes many components work together to achieve a common goal. It helps us understand the data in a geographic context, which enables to make better-informed decisions in a variety of fields such as urban planning, environmental management, and public health, among others that we will mention later.

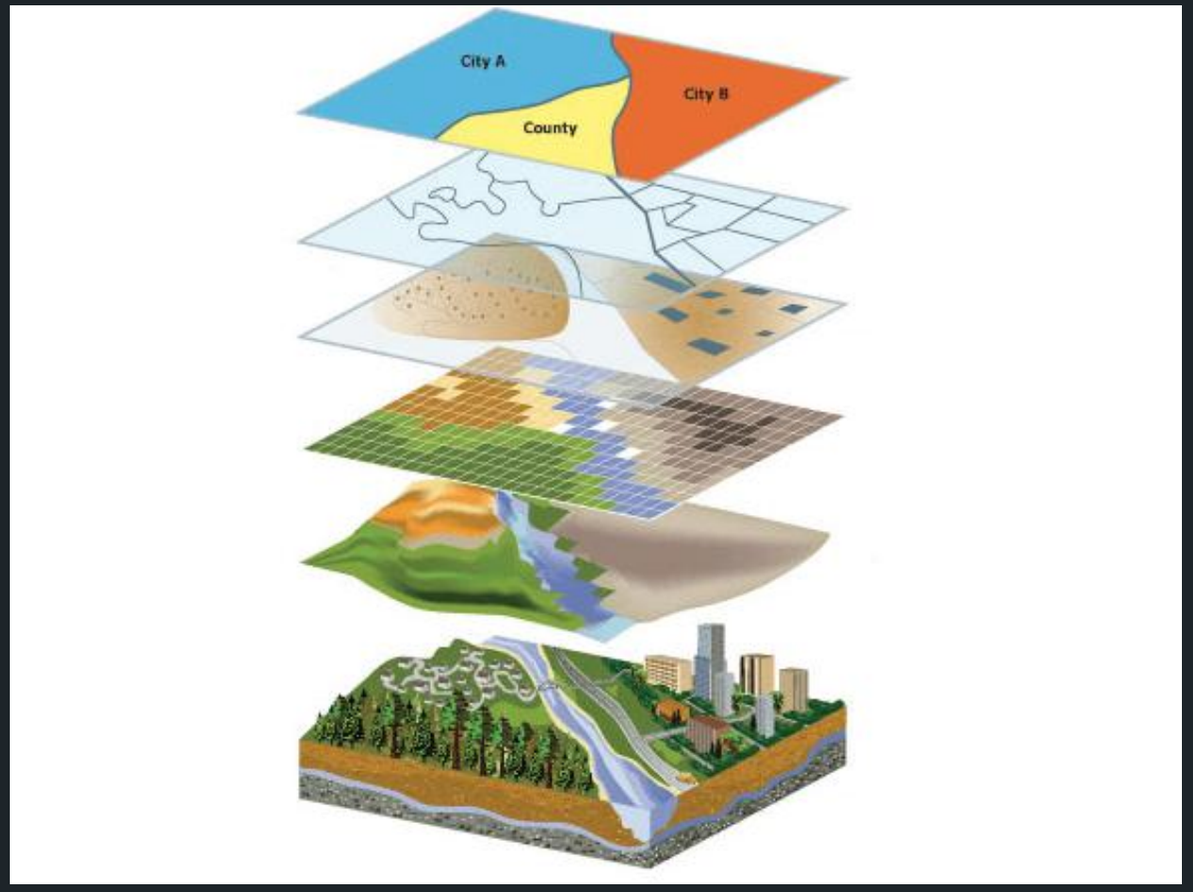


Source: GAO.

"...it is used by most industries. According to Data Bridge Market Research, Geographic information system market was valued at USD 9160.02 millionin 2021 and is expected to reach USD 22518.38 million by 2029.."

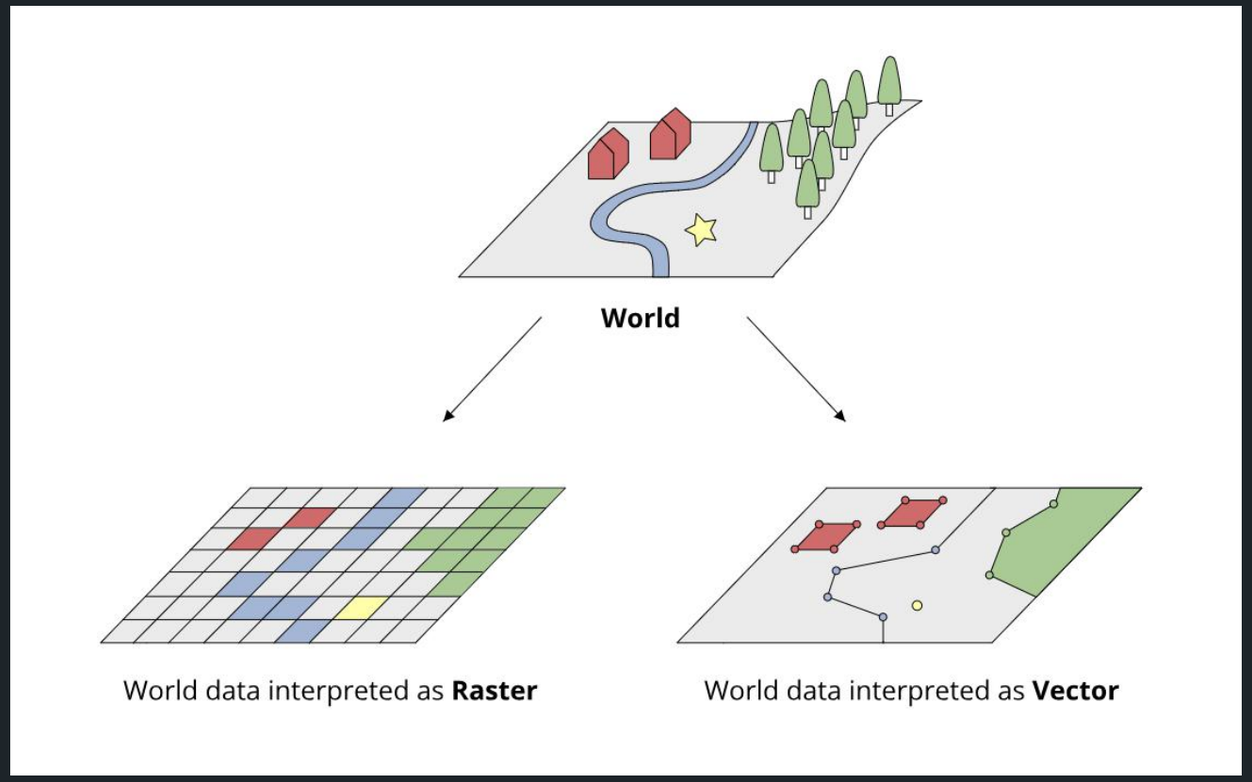
GEOGRAPHIC INFORMATION SYSTEMS (GIS)





We need a way to store, combine and integrate multiple types of geospatial data...

VECTOR AND RASTER DATA



Data Type	Key Characteristics	Common Formats	Examples
Raster	- Grid format (rows and columns) - Cell values for attributes - Resolution dependent	- GeoTIFF (.tif) - ERDAS IMAGINE (.img) - Digital Elevation Models (DEM)	- Satellite imagery - Elevation data (DEMs) - Land cover maps
Vector	- Shape-based (points, lines, polygons) - Precise location - Attribute richness	- Shapefile (.shp) - GeoJSON (.geojson) / TopoJSON (.topojson) - KML/KMZ (.kml/.kmz)	- Map of geological sample locations - River network diagrams - Boundary maps for geological formations

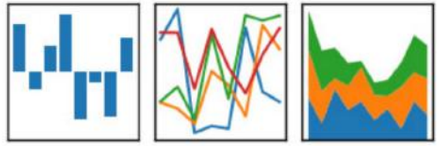
Some data files are human readable. e.g. GeoJSON below

```geojson
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [-73.9617, 40.6629]
      },
      "properties": {
        "name": "Brooklyn",
        "borough": "Kings",
        "state": "NY"
      }
    }
  ]
}
```

IN WALKS PANDAS....

pandas

$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$



"pandas...provides high-performance, easy-to-use data structures and data analysis tools.."

Format Type	Data Description	Reader	Writer
text	CSV	read_csv	to_csv
text	JSON	read_json	to_json
text	HTML	read_html	to_html
text	Local clipboard	read_clipboard	to_clipboard
binary	MS Excel	read_excel	to_excel
binary	HDF5 Format	read_hdf	to_hdf
binary	Feather Format	read_feather	to_feather
binary	Parquet Format	read_parquet	to_parquet
binary	Msgpack	read_msgpack	to_msgpack
binary	Stata	read_stata	to_stata
binary	SAS	read_sas	
binary	Python Pickle Format	read_pickle	to_pickle
SQL	SQL	read_sql	to_sql
SQL	Google Big Query	read_gbq	to_gbq

IN WALKS PANDAS....BUT, WE NEED A GEO EXTENSION...

GeoPandas

"pandas...provides high-performance, easy-to-use data structures and data analysis tools.."




```
e display
rt the modules, using standard naming convention.
import geopandas as gpd
import contextily as cx
import matplotlib.pyplot as plt
%matplotlib inline
import os
from os import path

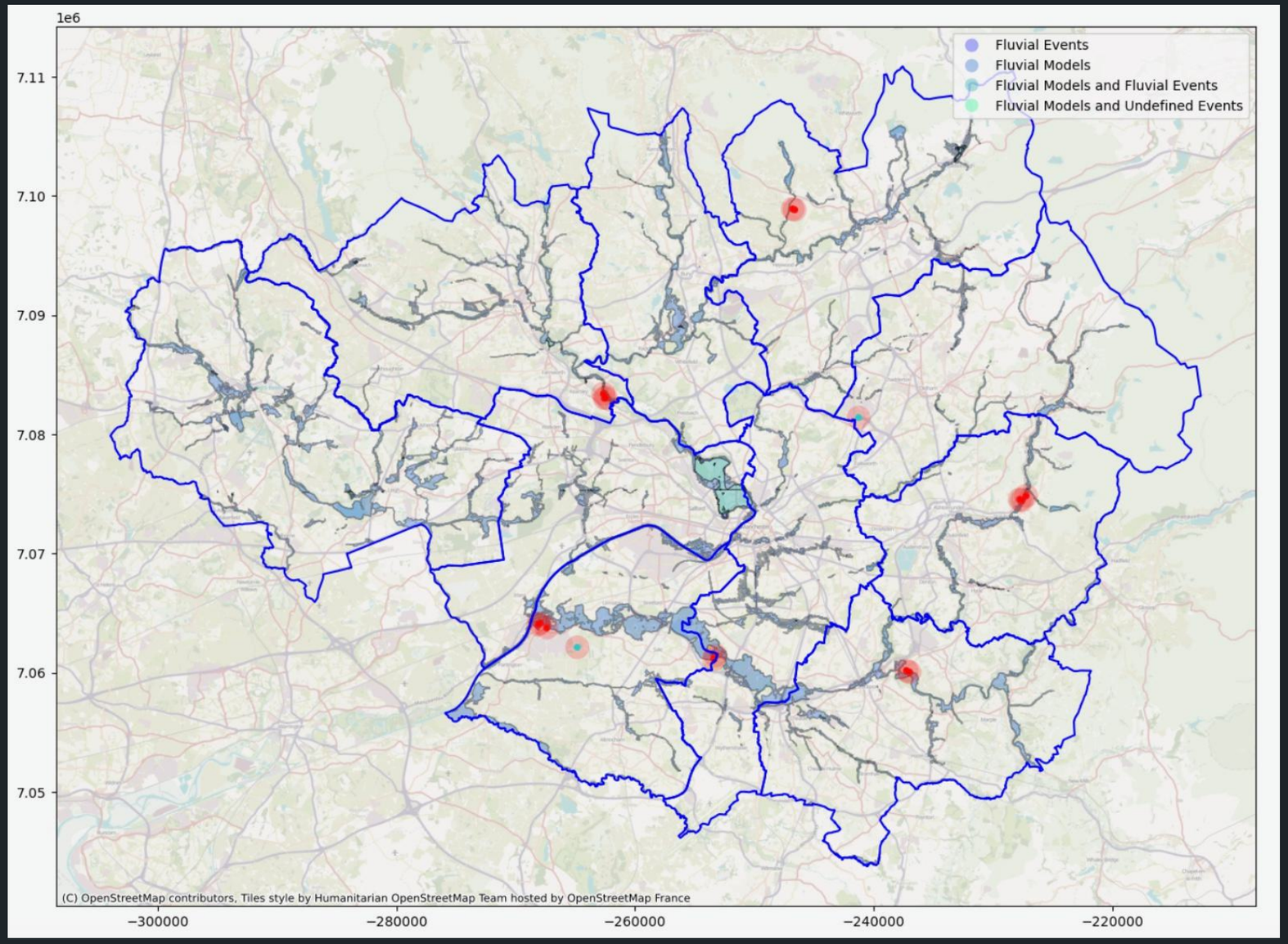
path=os.getcwd()

# Our file has the following path and name - notice the extension .shp
file = path+"/data/digmap625_bedrock_arc/625k_V5_BEDROCK_Geology_Polygons.shp"
# Like Pandas, GeoPandas has a read_file function. Im creating a GeoDataframe called 'gdf'
gdf = gpd.read_file(file)

# Preview the GeoDataframe to the screen
gdf
```

REF_GP	BGSREF_RK	SHEET	VERSION	RELEASED	NOM_SCALE	NOM_OS_YR	NOM_BGS_YR	MSLINK	geometry
773	773	625k_V5_Bedrock	5.17	11- 2-2008	625000	2005 HC	2007	18166	POLYGON ((274375.248 555042.401, 274315.421 55...
773	773	625k_V5_Bedrock	5.17	11- 2-2008	625000	2005 HC	2007	18166	POLYGON ((273618.809 553462.498, 273661.625 55...
773	773	625k_V5_Bedrock	5.17	11- 2-2008	625000	2005 HC	2007	18166	POLYGON ((270985.247 554515.841, 270985.247 55...
									POLYGON

WORKFLOW FOR TODAYS PRACTICAL - EXAMPLE OF FINAL FIGURE PRODUCED IN TODAYS NOTEBOOK



- Create data frames with geospatial data ✔
- Join polygon and point data
- Clip using regional boundaries
- Place all of this on a map
- Visualise:
  - risk by authority
  - distance between elements

BUT THE NUMBERS IN COLUMN GEOMETRY DO NOT LOOK LIKE COORDINATES??

- A Coordinate Reference System (CRS) defines how the two-dimensional, projected map in your GeoDataFrame relates to real places on the earth.
- CRS can be represented in various formats, such as EPSG codes.
- EPSG codes are short identifiers assigned to coordinate reference system (CRS) definitions. These codes are standardized and maintained by the EPSG (European Petroleum Survey Group) Geodetic Parameter Dataset.

EPSG Code	CRS Name	Description
EPSG:4326	WGS 84	Standard for global geographic coordinate system, used in GPS and for global datasets.
EPSG:3857	WGS 84 / Pseudo-Mercator	Also known as Web Mercator. Common for web mapping services like Google Maps and OpenStreetMap.

Correct CRS selection (via EPSG codes) is crucial for accurate distance and area measurements, especially over large geographic extents. This is crucial when combining data from different sources to ensure they align correctly.

BUT THE NUMBERS IN COLUMN GEOMETRY DO NOT LOOK LIKE COORDINATES??

```
gdf.crs
```

File display

```
<Projected CRS: PROJCS["OSGB36 / British National Grid",GEOGCS["OS ...>
Name: OSGB36 / British National Grid
Axis Info [cartesian]:
- [east]: Easting (metre)
- [north]: Northing (metre)
Area of Use:
- undefined
Coordinate Operation:
- name: unnamed
- method: Transverse Mercator
Datum: Ordnance Survey of Great Britain 1936
- Ellipsoid: Airy 1830
- Prime Meridian: Greenwich
```

As we combine data from different sources, we need to ensure they are based on the same CRS. Fortunately, GeoPandas allows us to move between different systems.

File display

```
gdf.to_crs(epsg=3857)
```

EF_GP	BGSREF_RK	SHEET	VERSION	RELEASED	NOM_SCALE	NOM_OS_YR	NOM_BGS_YR	MSLINK	geometry
773	773	625k_V5_Bedrock	5.17	11- 2-2008	625000	2005 HC	2007	18166	POLYGON ((-440750.842 7337513.465, -440854.565...
773	773	625k_V5_Bedrock	5.17	11- 2-2008	625000	2005 HC	2007	18166	POLYGON ((-441985.281 7334731.896, -441909.793...

PLOTTING VECTOR DATA ON A MAP

```
# Create an axis [ax1] that has the information held within the gdf_wm dataframe. Provide styling and colou
File display lf_wm.plot(figsize=(16, 16), alpha=0.3, column="RCS_D", edgecolor="k", cmap="gist_ncar",legend=True,
# Use the Contextily module to add a basemap underneath axes (ax1), which will look like OpenStreetMap. We
cx.add_basemap(ax1, crs='EPSG:3857')
```



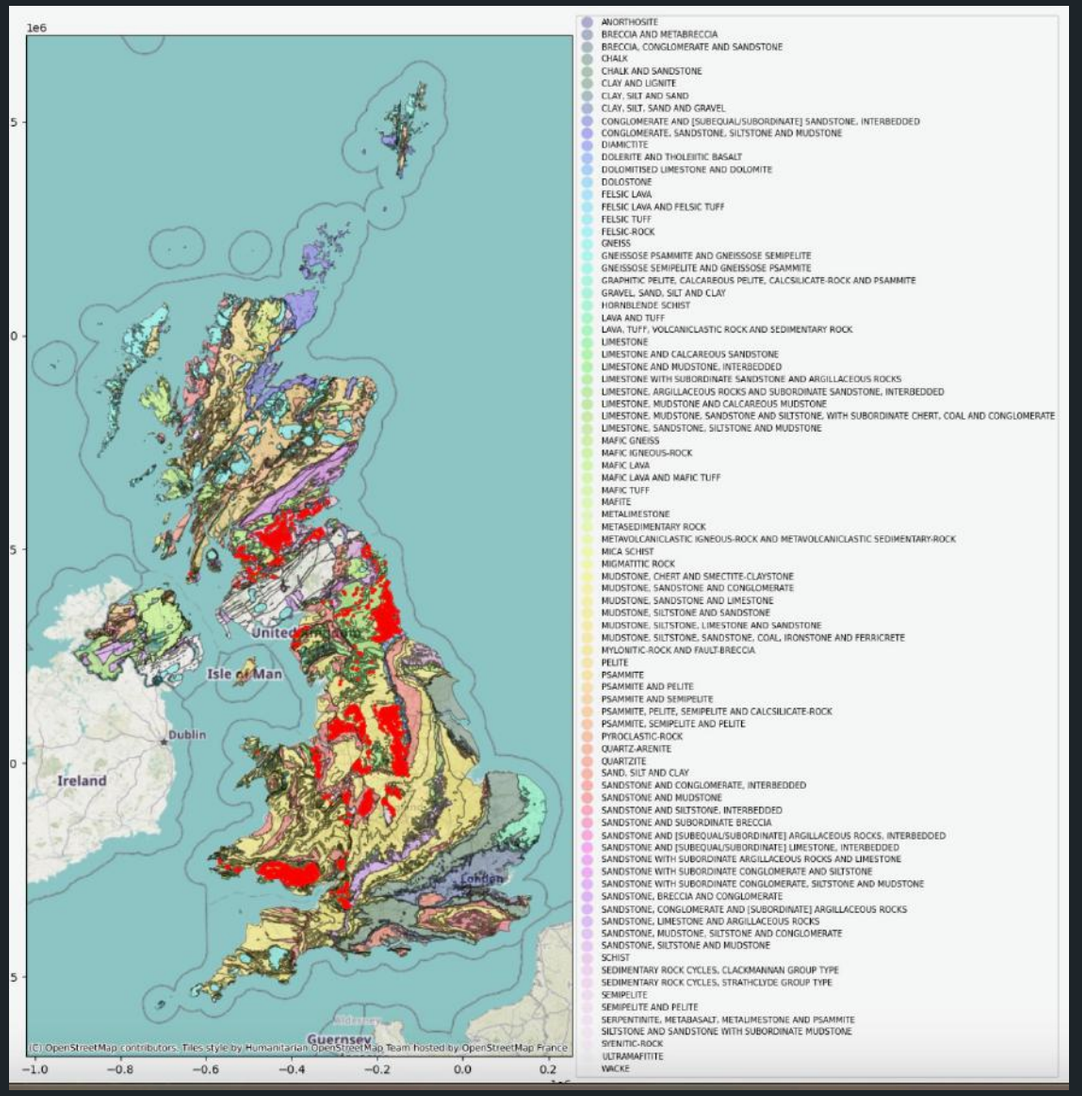
PLOTTING VECTOR AND RASTER DATA ON A MAP

```
# make referene to the location of the .shp file and then use GeoPandas to load in and then preview.
file_colliery = path+"/data/collieries/collieries.shp"
cdf = gpd.read_file(file_colliery)
cdf
```

osgb_east	osgb_north	latitude	longitude	notes	field_11	field_12	field_13	field_14	field_15	geometry
395255.499466851	283988.832690251	52.453832	-2.071245	None	None	None	None	None	None	POINT (-2.07124 52.45383)
400105.421757637	290519.531861927	52.512565	-1.999876	None	None	None	None	None	None	POINT (-1.99988 52.51256)
400022.412780917	290345.942248252	52.511004	-2.001100	None	None	None	None	None	None	POINT (-2.00110 52.51100)
400217.558344745	290229.001480458	52.509953	-1.998224	None	None	None	None	None	None	POINT (-1.99822 52.50995)

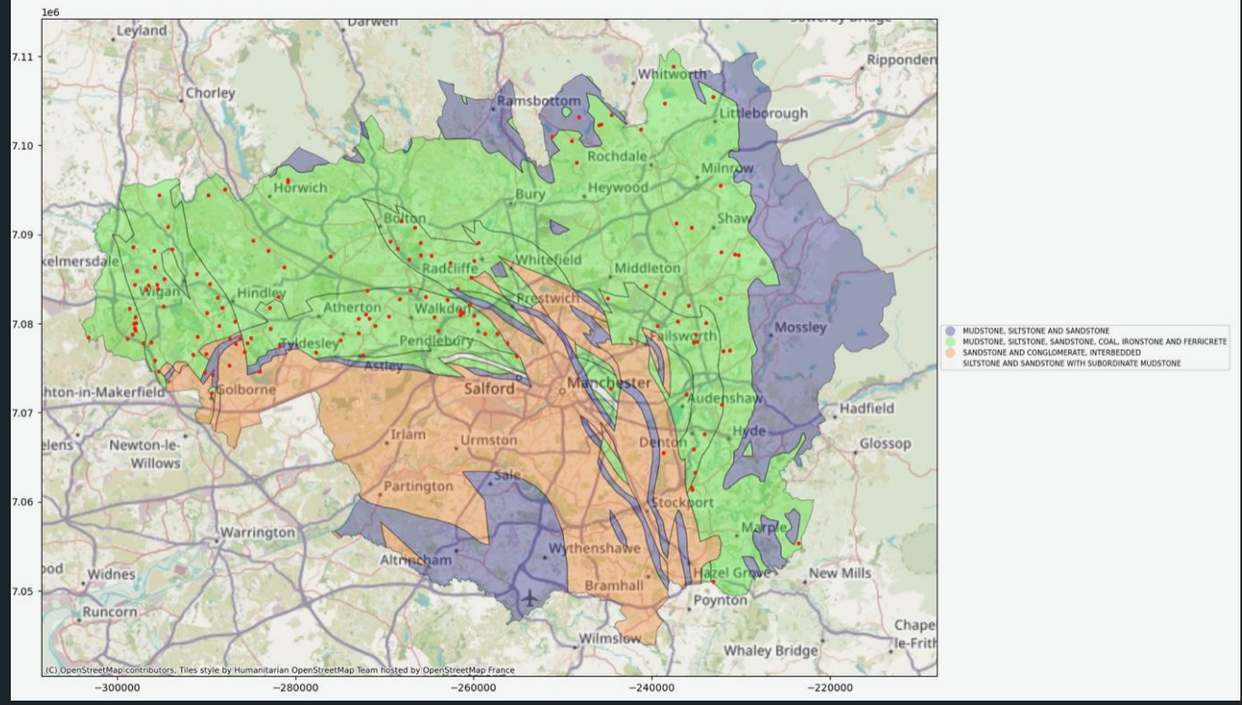
```
a new axis for our bedrock map, using the same options as before.
= gdf_wm.plot(figsize=(16, 16), alpha=0.3, column="RCS_D", edgecolor="k", cmap="gist_ncar", legend=True,
# Now plot data from our new colliery dataframr, cdf_wm, specifying the axis to host the datapoints.
cdf_wm.plot(ax=ax1_new, marker='o', color='red', markersize=5)
# Use contextily to add a basemap.
cx.add_basemap(ax1_new, crs='EPSG:3857')
```

File display



JOINING AND CLIP DATA FRAMES

```
# -------'INSERT CODE HERE'-------
# Location of the GM authority geometries
file_GM = path+"/data/gmauthorities.geojson"
# Read the .geojson file into a new dataframe
GMdf = gpd.read_file(file_GM)
# Convert to EPSG:3857
GMdf_wm = GMdf.to_crs(epsg=3857)
# Clip the bedrock dataframe
gdf_wm_GM = gdf_wm.clip(GMdf_wm)
# Clip the colliery dataframe
cdf_wm_GM=cdf_wm.clip(GMdf_wm)
# Create a map
ax_ex3 = gdf_wm_GM.plot(figsize=(16, 16), alpha=0.3, column="RCS_D", edgecolor="k", cmap="gist_ncar", legend=True
cdf_wm_GM.plot(ax=ax_ex3, marker='o', color='red', markersize=8)
#cx.add_basemap(ax3, source="basemap_GM.tif")
cx.add_basemap(ax_ex3, crs='EPSG:3857')
# --------------------------------
```



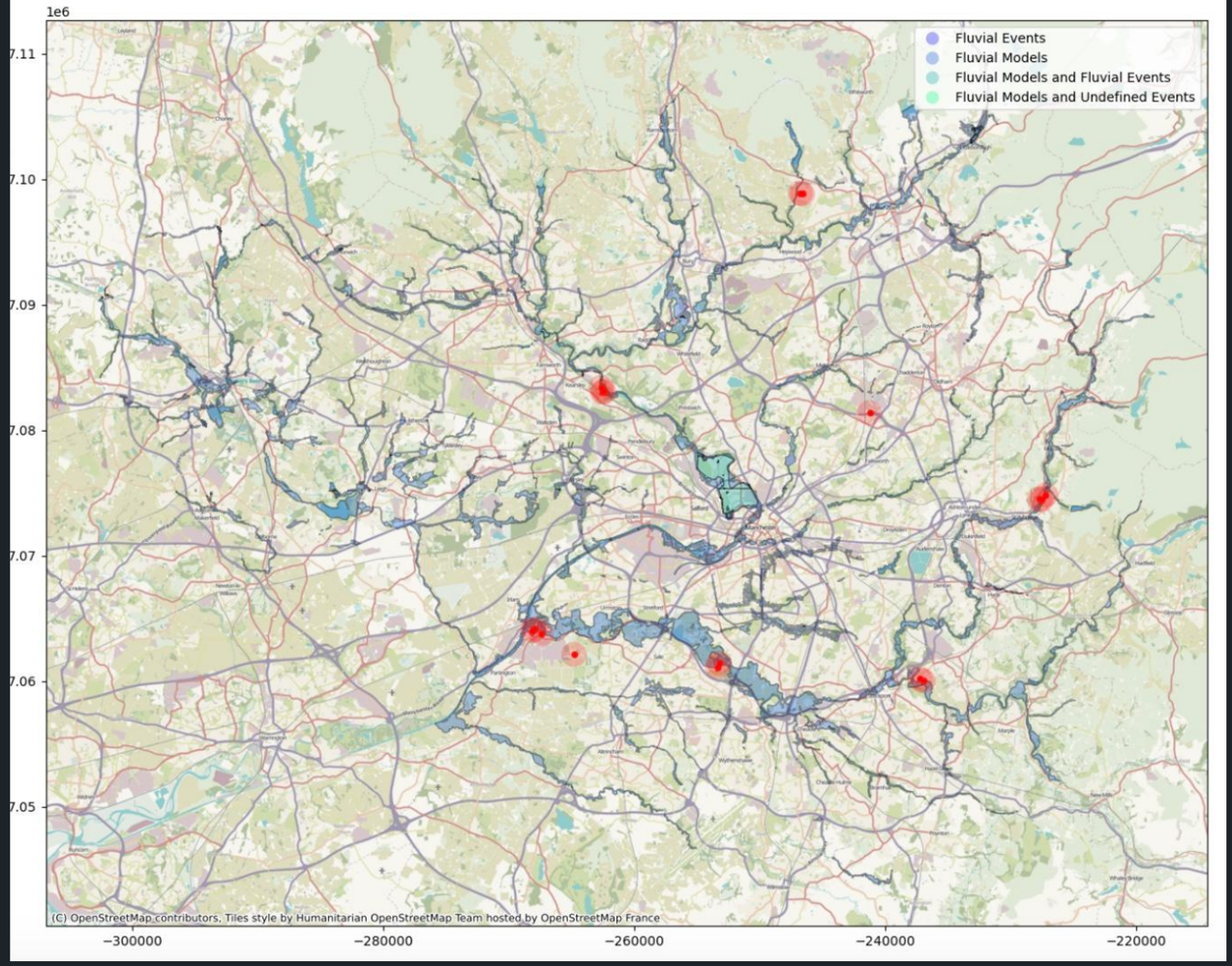
Join is a GeoPandas function that joins two data frames using their spatial data.
clip is a way of removing all data outside of a set of polygons
we can use both to produce a map where a subset of data is presented

DISTANCE FUNCTIONS

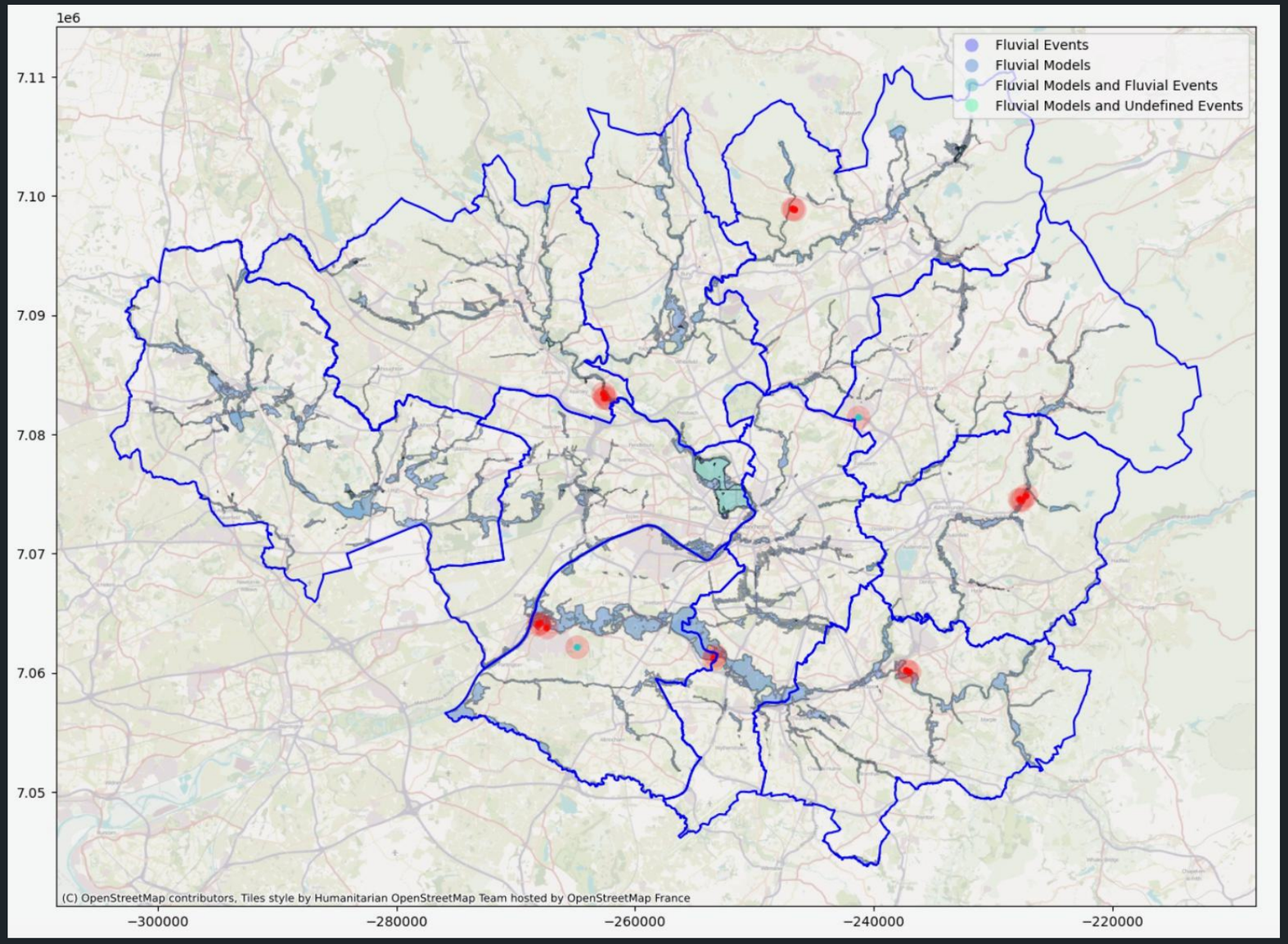
```
# get circle with 1 km radius
risk_zone = ssdf_wm_GM.buffer(1000)
risk_zone
```

```
389    POLYGON ((-235993.437 7060050.402, -235998.252...
174    POLYGON ((-236252.384 7060180.038, -236257.200...
393    POLYGON ((-252393.258 7061128.836, -252398.073...
407    POLYGON ((-252264.842 7061456.955, -252269.657...
16     POLYGON ((-226575.062 7074470.583, -226579.877...
492    POLYGON ((-226749.512 7074538.453, -226754.328...
387    POLYGON ((-263837.663 7062136.511, -263842.478...
209    POLYGON ((-266396.592 7063771.965, -266401.408...
60     POLYGON ((-267126.029 7064046.204, -267130.844...
59     POLYGON ((-266946.724 7064170.013, -266951.539...
137    POLYGON ((-226234.027 7074841.808, -226238.843...
320    POLYGON ((-240256.171 7081407.501, -240260.986...
31     POLYGON ((-261428.444 7083081.174, -261433.259...
62     POLYGON ((-245623.786 7098831.565, -245628.601...
219    POLYGON ((-245782.596 7098900.465, -245787.412...
223    POLYGON ((-261551.736 7083024.974, -261556.551...
225    POLYGON ((-261648.759 7083405.206, -261653.575...
dtype: geometry
```

```
ax_ex4d = frdf_wm_GM.plot(figsize=(16, 16), alpha=0.3, column="type", edgecolor="k", cmap="winter", legend=True)
ssdf_wm_GM.plot(ax=ax_ex4d, marker='o', color='red', markersize=18)
risk_zone.plot(ax=ax_ex4d, color="#ff000033")
cx.add_basemap(ax_ex4d, crs='EPSG:3857', zoom=12)
```

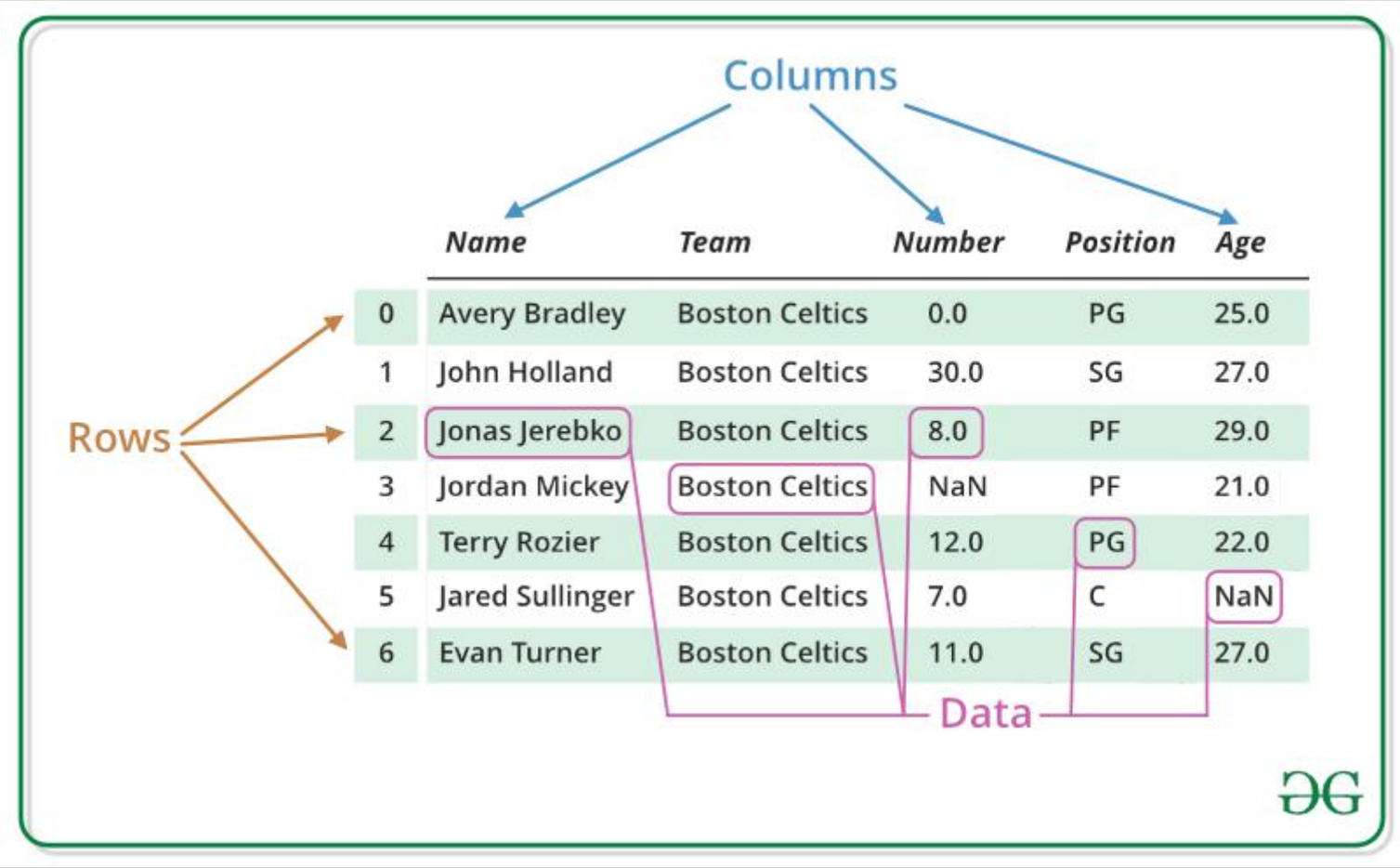


WORKFLOW FOR TODAYS PRACTICAL - EXAMPLE OF FINAL FIGURE PRODUCED IN TODAYS NOTEBOOK



- Create data frames with geospatial data
- Join polygon and point data
- Clip using regional boundaries
- Place all of this on a map
- Visualise:
  - risk by authority
  - distance between elements

IN WALKS GEOPANDAS



"*A GeoPandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns).*

A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. "

PANDAS AND [GEOSPATIAL] DATA DISCOVERY

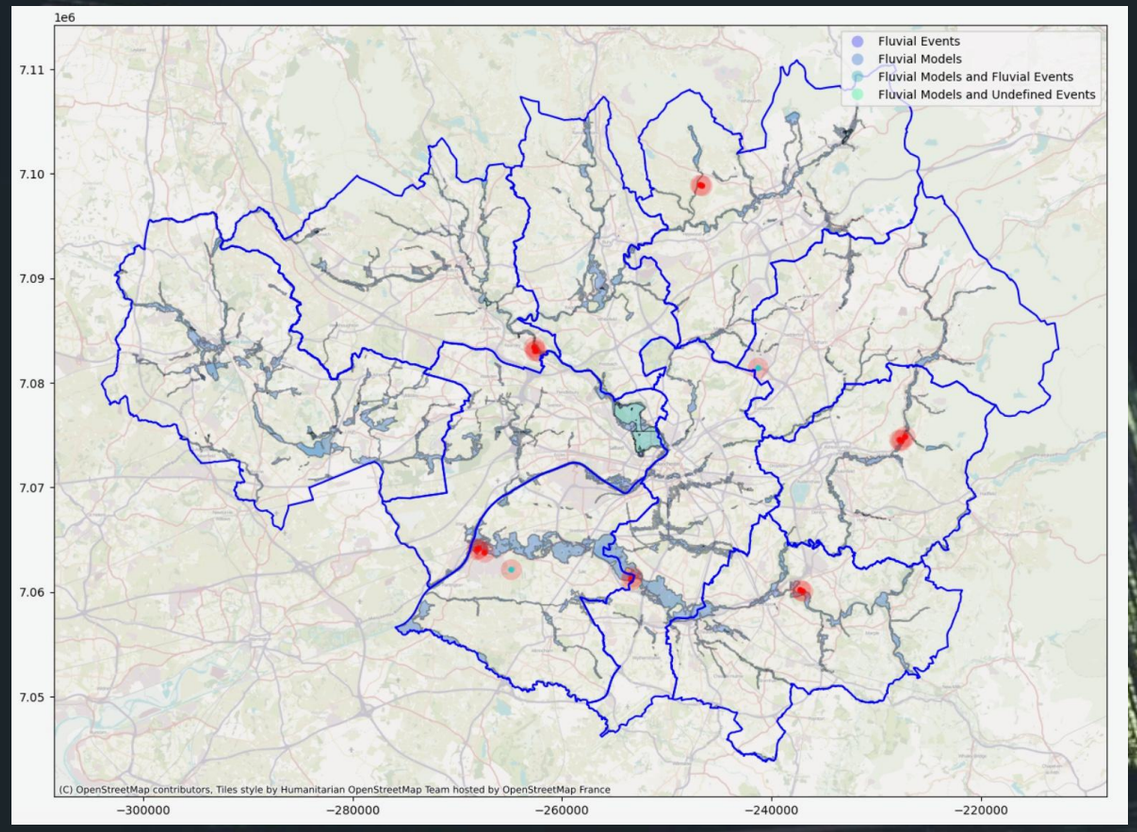


- DATA DATA EVERYWHERE.....IN WALKS PANDAS AND GEO-PANDAS
- PRACTICAL 7



David Topping: david.topping@manchester.ac.uk

LECTURE 7



SUMMARY

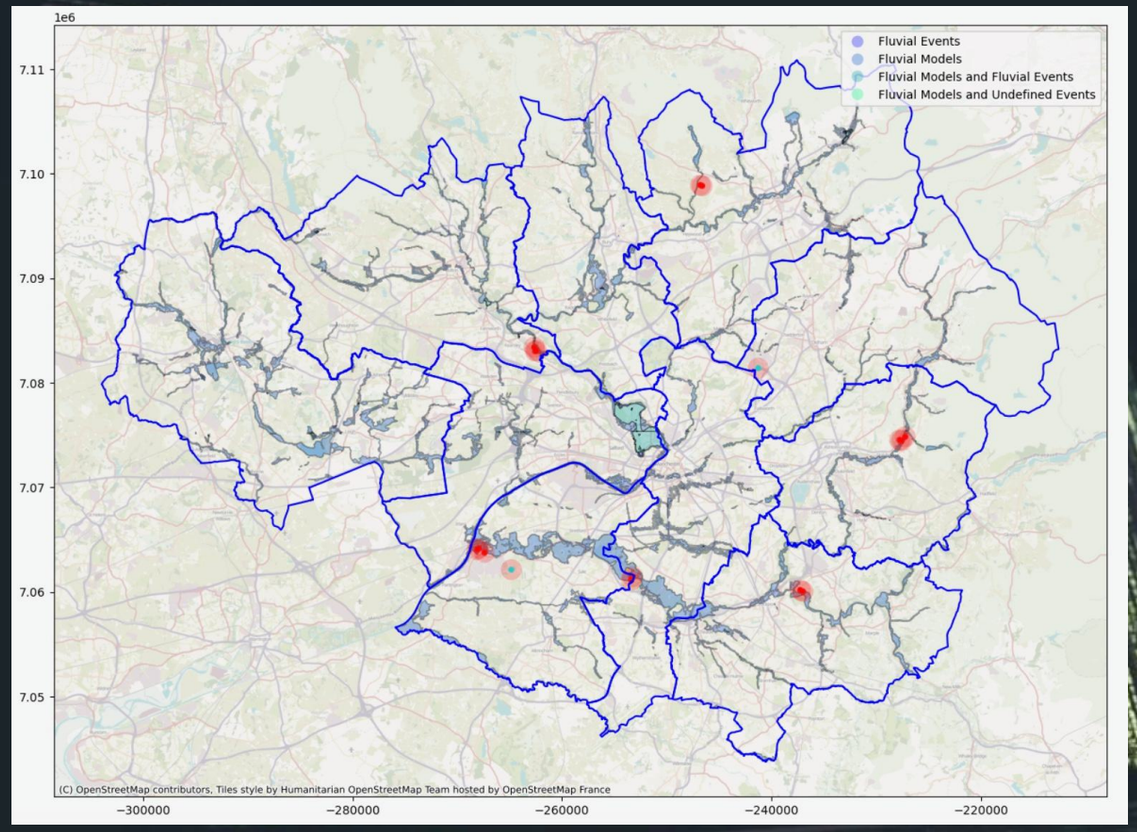
INTERACTING WITH GEOSPATIAL DATA IN PYTHON

- Data is everywhere.
- Geospatial data comes in vector and raster formats
- GeoPandas can load multiple file types
- Some formats are humn reasdable, others are not

GEOSPATIAL PLOTTING

- GeoPandas module allows us to plot both vector and raster data on the same map

PRACTICAL 7



GOALS

CODE

- Become familiar with GeoPandas
- Select a subset of data according to some criteria being met
- Plot vector and raster data

GENERAL EXPERIENCE

- Start to develop skills to interact with geospatial data - GiS highly valuable
- Increase the diversity of problems you can solve